

Convolución, Max Pooling y EdgeConv — Explicación Detallada

Para: Santiago (background Java/C++)

Contexto: DGCNN para predicción de vacancias

ÍNDICE

1. Convolución: Concepto General
 2. Max Pooling: Reducción de Dimensionalidad
 3. EdgeConv: El Corazón de DGCNN
 4. Forward Pass Completo con Ejemplos
-

1. CONVOLUCIÓN: CONCEPTO GENERAL

¿Qué es una convolución?

Una convolución es una **operación que extrae patrones locales** aplicando un filtro deslizante sobre los datos.

Analogía física (procesamiento de señales):

Imagina que tienes un micrófono que capta sonido. La convolución es como aplicar un filtro que:

- Amplifica ciertas frecuencias (ej: graves)
- Atenúa otras (ej: agudos)

En imágenes y point clouds, el filtro aprende a detectar **patrones geométricos**.

Convolución 1D (señales)

```
python
```

```
# Señal de entrada (1D)
```

```
signal = [1, 2, 3, 4, 5, 6, 7]
```

```
# Filtro/kernel (tamaño 3)
```

```
kernel = [0.5, 1.0, 0.5]
```

```
# Convolución: deslizar kernel sobre signal
```

```
output[0] = signal[0]*0.5 + signal[1]*1.0 + signal[2]*0.5 = 1*0.5 + 2*1.0 + 3*0.5 = 3.5
```

```
output[1] = signal[1]*0.5 + signal[2]*1.0 + signal[3]*0.5 = 2*0.5 + 3*1.0 + 4*0.5 = 5.0
```

```
output[2] = signal[2]*0.5 + signal[3]*1.0 + signal[4]*0.5 = 3*0.5 + 4*1.0 + 5*0.5 = 6.5
```

```
...
```

Efecto: suaviza la señal (promedio ponderado de vecinos)

Convolución 2D (imágenes)

```
python
```

```
# Imagen (5×5 píxeles, escala de grises)
```

```
image = [
```

```
    [10, 20, 30, 40, 50],
```

```
    [15, 25, 35, 45, 55],
```

```
    [20, 30, 40, 50, 60],
```

```
    [25, 35, 45, 55, 65],
```

```
    [30, 40, 50, 60, 70]
```

```
]
```

```
# Kernel detector de bordes (3×3)
```

```
kernel = [
```

```
    [-1, -1, -1],
```

```
    [-1, 8, -1],
```

```
    [-1, -1, -1]
```

```
]
```

```
# Convolución en posición (1,1):
```

```
output[1,1] = sum(image[0:3, 0:3] * kernel)
```

```
    = 10*(-1) + 20*(-1) + 30*(-1)
```

```
    + 15*(-1) + 25*(8) + 35*(-1)
```

```
    + 20*(-1) + 30*(-1) + 40*(-1)
```

```
    = -10 - 20 - 30 - 15 + 200 - 35 - 20 - 30 - 40
```

```
    = 0
```

Efecto: detecta cambios abruptos (bordes)

Convolución en Deep Learning

En redes neuronales, los **pesos del kernel se APRENDEN** durante el entrenamiento.

```
python

# Capa convolucional en PyTorch
conv = nn.Conv2d(
    in_channels=3,    # RGB (3 canales)
    out_channels=64,  # 64 filtros diferentes
    kernel_size=3     # Kernels de 3×3
)

# Cada uno de los 64 filtros aprende a detectar un patrón diferente:
# - Filtro 1: bordes horizontales
# - Filtro 2: bordes verticales
# - Filtro 3: esquinas
# - Filtro 4: texturas
# ... etc
```

Convolución en Point Clouds (nn.Conv2d en DGCNN)

En tu DGCNN usas `nn.Conv2d`, pero NO sobre imágenes 2D, sino sobre **grafos locales 4D**:

```
python

# Input shape: (B, C, N, k)
# B = batch size
# C = canales (features por punto)
# N = número de puntos
# k = vecinos por punto

# Ejemplo concreto:
edge_features = torch.randn(16, 6, 256, 20)
# 16 muestras
# 6 canales (n_channels=3 → 3*2=6 después de EdgeConv)
# 256 puntos
# 20 vecinos por punto

# Convolución 2D sobre las "imágenes" (C, k):
conv = nn.Conv2d(in_channels=6, out_channels=64, kernel_size=1)
output = conv(edge_features)
# Output: (16, 64, 256, 20)
```

¿Por qué kernel_size=1?

Porque la "imagen" es artificial (vecinos en feature space). Un kernel 1×1:

- Combina los 6 canales de entrada
- Genera 64 nuevos features
- **NO desliza espacialmente** (cada punto procesado independientemente)

Es equivalente a una **capa fully connected** aplicada a cada vecino.

2. MAX POOLING: REDUCCIÓN DE DIMENSIONALIDAD

¿Qué hace max pooling?

Selecciona el valor **MÁXIMO** de un grupo de valores.

Max Pooling 1D

```
python

input = [1, 4, 2, 7, 3, 9, 5, 6]

# Max pool con ventana de tamaño 2:
output[0] = max(1, 4) = 4
output[1] = max(2, 7) = 7
output[2] = max(3, 9) = 9
output[3] = max(5, 6) = 6

# Resultado: [4, 7, 9, 6]
# Tamaño reducido: 8 → 4
```

Max Pooling 2D (imágenes)

```
python
```

```
# Imagen 4×4
```

```
image = [  
    [1, 3, 2, 4],  
    [5, 6, 7, 8],  
    [9, 2, 3, 1],  
    [4, 5, 6, 7]  
]
```

```
# Max pool 2×2:
```

```
output[0,0] = max(1, 3, 5, 6) = 6  
output[0,1] = max(2, 4, 7, 8) = 8  
output[1,0] = max(9, 2, 4, 5) = 9  
output[1,1] = max(3, 1, 6, 7) = 7
```

```
# Resultado: [[6, 8],
```

```
#           [9, 7]]
```

```
# Tamaño: 4×4 → 2×2
```

Max Pooling en DGCNN

En tu código:

```
python
```

```
# Después de convolución en EdgeConv:
```

```
x = edge_features # (B, 64, N, k) = (16, 64, 256, 20)
```

```
# Max pool sobre la dimensión de vecinos (dim=-1):
```

```
x = x.max(dim=-1, keepdim=False)[0]
```

```
# Output: (16, 64, 256)
```

```
# ¿Qué pasó?
```

```
# Para cada punto, de sus 20 vecinos, se queda con la
```

```
# activación MÁXIMA de cada uno de los 64 filtros
```

Interpretación geométrica:

```
python

# Punto i tiene vecinos j ∈ {1, 2, ..., 20}
# Filtro f detecta "curvatura alta"

activations_filter_f = [0.1, 0.3, 0.9, 0.2, ...] # 20 valores

# Max pool:
max_activation = 0.9

# Significado: "En el vecindario del punto i,
#           AL MENOS UN vecino tiene curvatura alta"
```

Max pooling = detector de presencia de patrón (invariante al orden de vecinos).

¿Por qué max y no average?

Max pooling:

- Detecta **presencia** de feature (binario: "¿existe este patrón?")
- Invariante a permutaciones
- Robusto a ruido (outliers no dominan)

Average pooling:

- Captura **magnitud promedio**
- Puede diluir señales importantes
- Sensible a ruido

Tu DGCNN usa AMBOS:

```
python

# Global pooling (último paso):
x_max = F.adaptive_max_pool1d(x, 1) # Presencia de patrones
x_avg = F.adaptive_avg_pool1d(x, 1) # Distribución global
x = torch.cat((x_max, x_avg), 1) # Combina ambos
```

3. EDGECONV: EL CORAZÓN DE DGCNN

Concepto: Convolución en Grafos Dinámicos

EdgeConv es una operación que:

1. Construye un **grafo local** (k-NN) para cada punto
2. Calcula **edge features** entre punto y vecinos
3. Aprende **transformaciones** de esas edge features
4. **Agrega** (max pool) información de vecinos

Diferencia clave vs convolución tradicional:

- Convolución 2D: vecinos en grilla regular (píxeles)
 - EdgeConv: vecinos en espacio de features (dinámico)
-

PASO 1: Construcción del Grafo (k-NN)

```
python

def knn(x: torch.Tensor, k: int) -> torch.Tensor:
    """
    Encuentra k vecinos más cercanos para cada punto.
    x: (B, D, N) batch, dimensions, points
    """
    # Calcular distancias euclidianas entre todos los pares
    inner = -2 * torch.matmul(x.transpose(2, 1), x)
    xx = torch.sum(x ** 2, dim=1, keepdim=True)
    pairwise_distance = -xx - inner - xx.transpose(2, 1)

    # Seleccionar top-k más cercanos
    idx = pairwise_distance.topk(k=k, dim=-1)[1]
    return idx # (B, N, k)
```

Ejemplo numérico:

python

3 puntos en 2D:

```
points = torch.tensor([
    [[0.0, 0.0], # Punto 0
     [1.0, 0.0], # Punto 1
     [0.5, 1.0]] # Punto 2
]).transpose(1, 2) # (1, 2, 3)
```

Distancias:

```
dist[0,0] = 0    dist[0,1] = 1.0    dist[0,2] = 1.118
dist[1,0] = 1.0  dist[1,1] = 0      dist[1,2] = 1.118
dist[2,0] = 1.118 dist[2,1] = 1.118 dist[2,2] = 0
```

k=2 vecinos más cercanos:

```
knn_idx[0] = [0, 1] # Punto 0: él mismo + punto 1
knn_idx[1] = [1, 0] # Punto 1: él mismo + punto 0
knn_idx[2] = [2, 0] # Punto 2: él mismo + punto 0
```

¿Por qué incluir el propio punto?

Para que la feature del punto central también se procese (identidad en el grafo).

PASO 2: Edge Features

python

```
def get_graph_feature(x: torch.Tensor, k: int = 20) -> torch.Tensor:
    """
    Construye edge features: concatena (xj - xi, xi)
    """
    batch_size = x.size(0)
    num_points = x.size(2)

    # 1. Obtener índices k-NN
    idx = knn(x, k=k) # (B, N, k)

    # 2. Expandir índices para batches
    idx_base = torch.arange(0, batch_size, device=x.device).view(-1, 1, 1) * num_points
    idx = idx + idx_base
    idx = idx.view(-1)

    # 3. Recopilar features de vecinos
    _, num_dims, _ = x.size()
    x = x.transpose(2, 1).contiguous() # (B, N, D)

    # Flatten y gather
    feature = x.view(batch_size * num_points, -1)[idx, :]
    feature = feature.view(batch_size, num_points, k, num_dims)

    # 4. Repetir punto central
    x = x.view(batch_size, num_points, 1, num_dims).repeat(1, 1, k, 1)

    # 5. Concatenar (xj - xi, xi)
    feature = torch.cat((feature - x, x), dim=3) # (B, N, k, 2*D)
    feature = feature.permute(0, 3, 1, 2).contiguous() # (B, 2*D, N, k)

    return feature
```

Ejemplo numérico (simplificado):

python

Punto i en 3D:

```
xi = [0.5, 0.3, 0.8]
```

Sus k=3 vecinos:

```
neighbor_1 = [0.6, 0.4, 0.7]
```

```
neighbor_2 = [0.4, 0.2, 0.9]
```

```
neighbor_3 = [0.5, 0.5, 0.6]
```

Edge features para cada vecino:

```
edge_1 = concat([neighbor_1 - xi, xi])  
        = concat([0.1, 0.1, -0.1], [0.5, 0.3, 0.8])  
        = [0.1, 0.1, -0.1, 0.5, 0.3, 0.8] # 6D (2×3D)
```

```
edge_2 = concat([-0.1, -0.1, 0.1], [0.5, 0.3, 0.8])  
        = [-0.1, -0.1, 0.1, 0.5, 0.3, 0.8]
```

```
edge_3 = concat([0.0, 0.2, -0.2], [0.5, 0.3, 0.8])  
        = [0.0, 0.2, -0.2, 0.5, 0.3, 0.8]
```

Resultado: (3 vecinos, 6 dimensiones)

```
edge_features_i = [  
    [0.1, 0.1, -0.1, 0.5, 0.3, 0.8],  
    [-0.1, -0.1, 0.1, 0.5, 0.3, 0.8],  
    [0.0, 0.2, -0.2, 0.5, 0.3, 0.8]  
]
```

¿Por qué ($x_j - x_i$, x_i)?

- **$x_j - x_i$** : captura geometría **relativa** (¿el vecino está arriba/abajo/izquierda?)
- **x_i** : preserva contexto **absoluto** (¿dónde está el punto en el espacio global?)

Analogía: cómo describes la ubicación de tu casa:

- Relativa: "3 cuadras al norte del parque" ($x_j - x_i$)
- Absoluta: "Coordenadas GPS: -34.6037, -58.3816" (x_i)

PASO 3: Aprendizaje con Convolución

```
python

# Edge features: (B, 2*n_channels, N, k)
# Ejemplo: (16, 6, 256, 20) para n_channels=3, k=20

# Convolución 2D (kernel 1×1):
self.conv1 = nn.Sequential(
    nn.Conv2d(6, 64, kernel_size=1, bias=False),
    nn.BatchNorm2d(64),
    nn.LeakyReLU(negative_slope=0.2)
)

output = self.conv1(edge_features)
# Output: (16, 64, 256, 20)
```

¿Qué aprende la convolución?

Cada uno de los 64 filtros aprende a detectar un patrón geométrico local:

```
python

# Filtro 1: detecta "vecino muy cercano en x"
weights_f1 = [10.0, 0.0, 0.0, ...] # Peso alto en (xj-xi)_x

# Filtro 2: detecta "vecino alejado en cualquier dirección"
weights_f2 = [3.0, 3.0, 3.0, ...] # Pesos uniformes en (xj-xi)

# Filtro 3: detecta "punto central con coordenada z alta"
weights_f3 = [0.0, 0.0, 0.0, 0.0, 0.0, 8.0, ...] # Peso alto en xi_z
```

Después del entrenamiento, estos pesos se optimizan para distinguir las 9 categorías de vacancias.

PASO 4: Agregación (Max Pooling)

```
python

# Después de convolución:
x = output # (16, 64, 256, 20)

# Max pool sobre vecinos:
x = x.max(dim=-1, keepdim=False)[0]
# Output: (16, 64, 256)
```

Interpretación:

Para cada punto i:

- Procesamos sus 20 vecinos con 64 filtros
- Cada filtro genera 20 activaciones (una por vecino)
- Max pool: nos quedamos con la activación MÁXIMA

Significado:

"Para el punto i , ¿existe AL MENOS UN vecino que active el filtro f ?"

Ejemplo:

```
python

# Filtro f=12 detecta "curvatura alta"
activations_filter_12 = [0.1, 0.05, 0.95, 0.08, ...] # 20 valores

# Max pool:
max_activation = 0.95

# Interpretación:
# "Sí, el punto  $i$  tiene AL MENOS UN vecino con curvatura alta"
# (probablemente está cerca de una esquina o pico del nanoporo)
```

RESUMEN EdgeConv:

```
python

def EdgeConv(x, k=20):
    """
    x: (B, D_in, N) # Batch, dims_entrada, points
    Output: (B, D_out, N)
    """

    # 1. Construir grafo k-NN
    neighbors_idx = knn(x, k)

    # 2. Edge features: (xj - xi, xi)
    edge_feat = get_graph_feature(x, k) # (B, 2*D_in, N, k)

    # 3. Convolución (aprender transformación)
    edge_feat = Conv2d(2*D_in → D_out)(edge_feat) # (B, D_out, N, k)
    edge_feat = BatchNorm2d(edge_feat)
    edge_feat = LeakyReLU(edge_feat)

    # 4. Agregación (max pool)
    point_feat = edge_feat.max(dim=-1)[0] # (B, D_out, N)

    return point_feat
```

¿Por qué "dinámico"?

Porque el grafo k-NN se reconstruye en CADA capa:

```
python

# Capa 1: k-NN en espacio XYZ
x1 = EdgeConv(xyz_features) # Vecinos geométricos

# Capa 2: k-NN en espacio de features aprendidas
x2 = EdgeConv(x1) # Vecinos en feature space (ya NO xyz)

# Capa 3: k-NN en features más abstractas
x3 = EdgeConv(x2) # Vecinos según patrones aprendidos
```

Los vecinos cambian dinámicamente según la representación actual.

4. FORWARD PASS COMPLETO CON EJEMPLOS

Ejemplo concreto: 1 muestra, 256 puntos, 3 canales (XYZ)

Input:

```
python

point_cloud = torch.randn(1, 3, 256)
# 1 muestra
# 3 canales: [x, y, z] normalizadas en [-1, 1]
# 256 puntos de superficie del nanoporo
```

EdgeConv 1:

python

1. k-NN en espacio XYZ (k=20)

```
neighbors_idx = knn(point_cloud, k=20)
```

Para cada uno de los 256 puntos, encuentra sus 20 vecinos más cercanos

2. Edge features

```
edge_feat_1 = get_graph_feature(point_cloud, k=20)
```

Shape: (1, 6, 256, 20)

$6 = 2 \times 3$ canales (xj-xi: 3D, xi: 3D)

3. Convolución

```
self.conv1 = nn.Conv2d(6 → 64, kernel_size=1)
```

```
x1 = self.conv1(edge_feat_1)
```

Shape: (1, 64, 256, 20)

64 filtros aprendidos

4. Max pooling

```
x1 = x1.max(dim=-1)[0]
```

Shape: (1, 64, 256)

Por cada punto, la activación máxima entre sus 20 vecinos

Interpretación física:

- Punto i está en una **esquina del nanoporo**
- Sus 20 vecinos forman una **geometría angular**
- Filtro f=7 detecta "esquinas" → activación alta
- Max pool: $x1[0, 7, i] = 0.95$ (esquina detectada)

EdgeConv 2:

```
python
```

```
# Ahora x1 = (1, 64, 256) es el input
```

```
# 1. k-NN en feature space (64D)
```

```
neighbors_idx = knn(x1, k=20)
```

```
# Los 20 vecinos YA NO son los mismos que en EdgeConv1
```

```
# Ahora son vecinos en el espacio de features aprendidas
```

```
# 2. Edge features
```

```
edge_feat_2 = get_graph_feature(x1, k=20)
```

```
# Shape: (1, 128, 256, 20)
```

```
# 128 = 2 × 64 canales
```

```
# 3. Convolución
```

```
self.conv2 = nn.Conv2d(128 → 64, kernel_size=1)
```

```
x2 = self.conv2(edge_feat_2)
```

```
# Shape: (1, 64, 256, 20)
```

```
# 4. Max pooling
```

```
x2 = x2.max(dim=-1)[0]
```

```
# Shape: (1, 64, 256)
```

¿Qué cambió?

En EdgeConv1:

- Vecinos: puntos geoméricamente cercanos en XYZ

En EdgeConv2:

- Vecinos: puntos con **features similares**
- Ejemplo: todos los puntos en "esquinas" se agrupan
- Aunque estén LEJOS geoméricamente

Esto es el "grafo dinámico": la topología cambia según las features.

EdgeConv 3 y 4:

```
python
```

```
# EdgeConv 3: (1, 64, 256) → (1, 128, 256)
```

```
x3 = EdgeConv(x2, conv: 128 → 128)
```

```
# EdgeConv 4: (1, 128, 256) → (1, 256, 256)
```

```
x4 = EdgeConv(x3, conv: 256 → 256)
```

Cada capa aprende representaciones más abstractas:

- Layer 1: geometría local (curvaturas, planos)
 - Layer 2: patrones de superficie (pliegues, cavidades)
 - Layer 3: topología intermedia (conectividad)
 - Layer 4: estructura global (forma completa del nanoporo)
-

Concatenación Multi-Escala:

```
python

# Combinar features de todas las capas:
x = torch.cat((x1, x2, x3, x4), dim=1)
# Shape: (1, 64+64+128+256, 256) = (1, 512, 256)
```

¿Por qué concatenar?

Captura información a **múltiples escalas** simultáneamente:

- x1: detalles finos (geometría punto a punto)
- x2: patrones locales
- x3: estructuras intermedias
- x4: forma global

Similar a "skip connections" en ResNet.

Global Pooling:

python

Convolución adicional para emb_dims

```
self.conv5 = nn.Conv1d(512 → 512, kernel_size=1)
```

```
x = self.conv5(x) # (1, 512, 256)
```

Max pooling global (sobre todos los puntos)

```
x_max = F.adaptive_max_pool1d(x, 1)
```

Shape: (1, 512, 1) → (1, 512)

Para cada filtro: ¿cuál punto lo activó más?

Avg pooling global

```
x_avg = F.adaptive_avg_pool1d(x, 1)
```

Shape: (1, 512, 1) → (1, 512)

Para cada filtro: activación promedio

Concatenar

```
x_global = torch.cat((x_max, x_avg), dim=1)
```

Shape: (1, 1024)

Vector de 1024 features globales:

python

```
x_global[0:512]: # Max pooling → presencia de patrones
```

```
x_global[512:1024]: # Avg pooling → distribución global
```

Ejemplo de interpretación:

```
x_global[37] = 0.95 # Filtro 37 activado fuertemente (max)
```

→ Detectó esquinas agudas

```
x_global[512+37] = 0.23 # Activación promedio baja
```

→ Pocas esquinas, geometría mayormente suave

Clasificador MLP:

```
python
```

```
# Linear 1
```

```
x = self.linear1(x_global) # (1, 1024) → (1, 256)
```

```
x = self.bn1(x)           # Batch normalization
```

```
x = F.leaky_relu(x)
```

```
x = self.dp1(x)           # Dropout 0.4
```

```
# Linear 2
```

```
x = self.linear2(x)       # (1, 256) → (1, 128)
```

```
x = self.bn2(x)
```

```
x = F.leaky_relu(x)
```

```
x = self.dp2(x)
```

```
# Linear 3 (clasificación)
```

```
logits = self.linear3(x)  # (1, 128) → (1, 9)
```

Logits (antes de softmax):

```
python
```

```
logits = [
```

```
    -2.3, # point_defect
```

```
    -1.5, # small
```

```
    0.8, # medium_small
```

```
    3.2, # medium_large ← MÁXIMO
```

```
    1.1, # large_small
```

```
    -0.5, # extra_large_1
```

```
    -1.8, # extra_large_2
```

```
    -2.1, # extra_large_3
```

```
    -2.7 # extra_large_4
```

```
]
```

```
# Predicción: argmax(logits) = 3 → medium_large
```

```
predicted_class = 3
```

```
confidence = softmax(logits)[3] = 0.87 # 87%
```

RESUMEN CONCEPTUAL

Convolución:

- **Qué:** Aplicar filtros aprendidos para detectar patrones
- **Dónde:** En edge features (B, 2*D, N, k)
- **Cómo:** Combina canales de entrada → genera nuevas features
- **kernel_size=1:** transformación punto a punto (no espacial)

Max Pooling:

- **Qué:** Seleccionar activación máxima
- **Dónde:** Sobre dimensión de vecinos (k)
- **Cómo:** Para cada punto, de sus k vecinos $\rightarrow \max(\text{activations})$
- **Efecto:** Invariancia a orden, detección de presencia

EdgeConv:

- **Qué:** Operación graph-based que aprende de vecindarios locales
- **Cómo:**
 1. k-NN $\rightarrow$ grafo local
 2. Edge features $\rightarrow (x_j - x_i, x_i)$
 3. Conv2d $\rightarrow$ aprende transformación
 4. Max pool $\rightarrow$ agrega información
- **Dinámico:** Grafo cambia cada capa según features



EJEMPLO VISUAL: UN NANOPORO COMPLETO

Imaginemos un nanoporo con **geometría compleja**:

Input Point Cloud (256 puntos):

- 100 puntos en superficie SUAVE (lado A)
- 50 puntos en ESQUINA aguda (vértice)
- 106 puntos en superficie ONDULADA (lado B)

EdgeConv 1 (espacio XYZ):

```
python
```

```
# Punto i en superficie suave:
```

```
neighbors_i = [puntos cercanos geométricamente]
                $\rightarrow$  todos en superficie plana
```

```
edge_features_i = pequeñas diferencias
```

```
activations[filtro_esquina] = BAJO (0.1)
```

```
activations[filtro_plano] = ALTO (0.9)
```

```
# Punto j en esquina:
```

```
neighbors_j = [puntos formando ángulo agudo]
```

```
edge_features_j = grandes diferencias direccionales
```

```
activations[filtro_esquina] = ALTO (0.95)
```

```
activations[filtro_plano] = BAJO (0.05)
```

EdgeConv 2 (feature space):

```
python

# Ahora en 64D feature space:
# Los 100 puntos de superficie suave tienen features similares
# → Se agrupan en k-NN aunque estén lejos en XYZ

# Los 50 puntos de esquina también se agrupan entre sí
# → Aprenden "contexto de esquina"
```

Global Pooling:

```
python

x_max[filtro_esquina] = 0.95
# "Sí, este nanoporo tiene AL MENOS una esquina aguda"

x_avg[filtro_esquina] = 0.25
# "Pero EN PROMEDIO, la geometría no es muy angular"
# (50 puntos angular / 256 total = ~20%)

# Clasificador interpreta:
# - Tiene algunas esquinas (max alto)
# - Pero mayoría suave (avg bajo)
# → Probablemente medium_large o large_small
```

💡 PREGUNTAS FRECUENTES

P: ¿Por qué no usar solo las coordenadas XYZ directamente en un MLP?

R: MLP no captura relaciones espaciales:

```
python

# MLP procesa cada punto independientemente:
for point in points:
    features = MLP(point) # No ve vecinos

# DGCNN ve CONTEXTO:
for point in points:
    neighbors = knn(point, k=20)
    features = EdgeConv(point, neighbors) # Ve geometría local
```

P: ¿Por qué $(x_j - x_i, x_i)$ y no solo $(x_j - x_i)$?

R: Solo diferencias pierde información absoluta:

```
python
```

```
# Dos casos indistinguibles con solo (xj-xi):
```

```
Case A: xi=[0,0,0], xj=[1,0,0] → diff=[1,0,0]
```

```
Case B: xi=[5,5,5], xj=[6,5,5] → diff=[1,0,0]
```

```
# Con (diff, xi):
```

```
Case A: [1,0,0, 0,0,0]
```

```
Case B: [1,0,0, 5,5,5] ← DIFERENTES
```

P: ¿k=20 es suficiente?

R: Depende de densidad de puntos:

```
python
```

```
# Para 256 puntos uniformes en esfera:
```

```
avg_distance_to_20th_neighbor ≈ 0.2 (en [-1,1] normalizado)
```

```
# Captura ~2-3 "capas" de átomos vecinos en la superficie
```

```
# Suficiente para detectar patrones locales
```

P: ¿Por qué LeakyReLU y no ReLU?

R: LeakyReLU evita "dead neurons":

```
python
```

```
# ReLU:
```

```
f(x) = max(0, x)
```

```
# Si x < 0 → gradient = 0 → neurona "muere"
```

```
# LeakyReLU:
```

```
f(x) = max(0.2*x, x)
```

```
# Si x < 0 → gradient = 0.2 → neurona sigue aprendiendo
```

CONCLUSIÓN

Tu DGCNN aprende automáticamente a:

1. **Detectar patrones geométricos** (convolución)
2. **Agregar información local** (max pooling)
3. **Construir representaciones jerárquicas** (EdgeConv dinámico)
4. **Clasificar nanoporos** en 9 categorías

Todo esto **sin features manuales** — la red aprende qué patrones importan para distinguir 1-3 vacancias vs 100+ vacancias.

¡Es deep learning aplicado a geometría 3D! 🎯